tmx-orchestrator

Revision: 3 **Last modified:** 2026-06-16T15:40:00Z

A real, compile-correct Go consumer binary that drives the `digital.vasic.containers` submodule library to orchestrate container distribution to remote test hosts (e.g. `nezha.local`) for the tmux project's testing needs.

Purpose

When the tmux project needs heavy/containerized test workloads run on a remote host instead of the developer's laptop, this binary is the on-demand entry point. It:

1. Loads the remote-host configuration from `Containers/.env`.
2. Auto-detects the local container runtime, sets up the SSH executor + host manager + resource-aware scheduler + distributor — the exact wiring proven in `Containers/cmd/boot/main.go`.
3. Schedules a container onto the best remote host, runs it over SSH, and confirms it with a real health check.

This binary lives entirely in the **tmux (consumer) project** and only **imports** the Containers library. It adds **no** tmux-specific code INTO the submodule — the submodule stays 100% decoupled (CONST-051).

How it consumes the Containers submodule

Containers package	Used for
`pkg/envconfig`	`LoadFromFile` → `DistributionConfig` → `ToRemoteHosts()`
`pkg/runtime`	`AutoDetect(ctx)` for the local runtime
`pkg/remote`	`NewSSHExecutor`, `NewHostManager`, `AddHost`, `ProbeHost`, `ListHosts`, `GetHost`
`pkg/scheduler`	`NewScheduler` + `WithStrategy`, `ContainerRequirements`
`pkg/distribution`	`NewDistributor` (+ `With*` options), `Distribute`, `Undistribute`, `DistributionSummary`
`pkg/health`	

Containers package	Used for
	`NewDefaultChecker` + `HealthTarget{Type: HealthTCP|HealthHTTP}` + `Check(ctx)`
`pkg/logging`	`NewStdLogger("tmx-orchestrator")`

The dependency is wired via the module `replace` directive in `go.mod`:

```
require digital.vasic.containers
v0.0.0-00010101000000-000000000000
replace digital.vasic.containers => ../../Containers
```

Build

```
cd scripts/tmx-orchestrator
go build -o ../tmx-orchestrator-bin .
```

The compiled binary `scripts/tmx-orchestrator-bin` is **gitignored** and regenerated via `go build` on each host (§11.4.77). The module source (`go.mod`, `go.sum`, `main.go`, `README.md`) IS tracked.

Usage

Run from the `scripts/tmx-orchestrator/` directory so the default `../../Containers/.env` resolution works, or pass `--env` explicitly.

`hosts` — register + probe every configured remote host

```
./tmx-orchestrator-bin hosts
./tmx-orchestrator-bin hosts --env ../../Containers/.env
```

Registers every `CONTAINERS_REMOTE_HOST_N_*` host, probes each for reachability + CPU/memory, and prints a table. **Exits non-zero if any configured host is unreachable.**

`distribute` — run a container on a remote host + health-check it

```
# TCP health check (default image nginx:alpine, name tmx-orch-
        demo, port 80)
./tmx-orchestrator-bin distribute

# HTTP health check against a deployed web container
./tmx-orchestrator-bin distribute \
    --image docker.io/library/nginx:alpine \
    --name tmx-web --port 80 --health http --health-path /

# TCP health check against a deployed redis container
./tmx-orchestrator-bin distribute \
```

```
    --image docker.io/library/redis:7 \
    --name tmx-redis --port 6379 --health tcp

# Publish container port 80 to host port 18080 on the remote,
        HTTP-check it
./tmx-orchestrator-bin distribute \
    --image docker.io/library/nginx:alpine \
    --name tmx-orch-demo --port 80 --publish 18080 \
    --health http --health-path /
```

Builds a `ContainerRequirements`, calls `Distributor.Distribute(...)` to place + run the container on the scheduled remote host, prints the `DistributionSummary` (local/remote/failed counts + per-container host placement), then runs the chosen health checker against the deployed host:port and prints the result. **Exits non-zero on distribution failure OR health-check failure.** This is the anti-bluff core — it actually runs a real container on the remote and confirms it via the health check.

Flags: `--image` (default `docker.io/library/nginx:alpine`), `--name` (default `tmx-orch-demo`), `--port` (container port, default `80`), `--publish` (host port to publish `--port` to; `0` = same as `--port`), `--health` (`tcp`|`http`, default `tcp`), `--health-path` (http only, default `/`).

Port publishing is required for a cross-host health check to reach the service. The remote deploy runs `<runtime> run -d --name X -p host:container image` (the `Ports` field of `ContainerRequirements`, added to the Containers library per §11.4.76); without it the container port lives only inside the container's network namespace and the conductor cannot reach it. The health check polls (~15 s) so a freshly-started service that is not yet accepting connections is given time to become ready (no false PASS — a genuinely-down service stays UNHEALTHY for the full window).

> **Health-check strength (`tcp` vs `http`).** A `--health tcp` check is *connect-only*: against a **published** port it is satisfied by the runtime's port-forward host listener even if the container itself is not serving that port — i.e. it confirms the port is published, not that the service responds. Prefer `--health http` for HTTP services: the request is forwarded through to the container, so a non-serving container yields `UNHEALTHY` (verified).

On a failed health check the container is intentionally **left running** on its host for inspection (logs/exec); the command prints a hint to run `down --name <name>` so a failed deploy never silently leaks a container.

down — tear down distributed container(s)

```
./tmx-orchestrator-bin down --name tmx-web
```

Removes the named container on every registered remote host and calls `Distributor.Undistribute(...)` to close tunnels / unmount volumes. Idempotent.

Common flags

- `--env <path>` — path to the `.env` config. Default search order: `../../Containers/.env`, `../../.env`, `./.env`, `$PWD/.env`.
- `--timeout <dur>` — overall context timeout (default `3m`), e.g. `90s`, `5m`. SIGINT/SIGTERM cancels the context.

.env schema

The config schema is owned by the Containers submodule — `Containers/pkg/envconfig/parser.go`. The operator-placed config lives at `Containers/.env` and registers `nezha.local`:

```
CONTAINERS_REMOTE_ENABLED=true
CONTAINERS_REMOTE_SCHEDULER=resource_aware
CONTAINERS_REMOTE_DEFAULT_SSH_USER=milosvasic
CONTAINERS_REMOTE_DEFAULT_SSH_KEY=~/.ssh/id_ed25519
CONTAINERS_REMOTE_DEFAULT_RUNTIME=podman

CONTAINERS_REMOTE_HOST_1_NAME=nezha
CONTAINERS_REMOTE_HOST_1_ADDRESS=nezha.local
CONTAINERS_REMOTE_HOST_1_PORT=22
CONTAINERS_REMOTE_HOST_1_USER=milosvasic
CONTAINERS_REMOTE_HOST_1_KEY=~/.ssh/id_ed25519
CONTAINERS_REMOTE_HOST_1_RUNTIME=podman
CONTAINERS_REMOTE_HOST_1_LABELS=arch=amd64,podman_available=true,rc
test
```

Hosts scale freely: append a `CONTAINERS_REMOTE_HOST_N_*` block (the loader stops at the first absent `_NAME`). `.env` is gitignored (CONST-053 / §11.4.10).

> **Key path:** use an ABSOLUTE path (e.g. `/Users/<you>/.ssh/id_ed25519`) for `*_KEY` on the conductor host — the SSH-exec path does not expand a leading `~`.

Related

- `Containers/cmd/boot/main.go` — the proven wiring this binary mirrors.
- `Containers/CLAUDE.md` — Composition + Remote Distribution sections.
- `Containers/pkg/envconfig/parser.go` — `.env` schema source of truth.